

RPG 40K Survivor

Assistant solo textuel inspire de Warhammer 40K : incarnez un citoyen imperial pris dans l'enfer d'une invasion tyranide. Le projet fournit :

- un prompt MJ grimdark pret a l'emploi,
- une fiche de personnage basique,
- un script CLI qui orchestre la partie avec l'API OpenAI,
- des utilitaires pour les jets de des, le suivi d'etat et les sauvegardes par scene,
- des tests simples pour s'assurer que la logique auxiliaire fonctionne.

Livrables module Coordination Front & Back (M2)

Ce projet est aussi le rendu du module **Coordination dev Front & Back**. Les livrables attendus par la grille sont regroupés dans [docs/module/](#) :

- [Document de cadrage](#) (objectifs, périmètre, stack, IA)
- [MCD / MLD](#) (modèle de données)
- [Documentation technique](#) (architecture, API, sécurité)
- [Wireframes](#) (écrans clés)
- [Checklist activités 1 à 10](#)
- [Audit technique et optimisation](#)
- [Sprint de finalisation et playtest](#)
- [Analyse critique — Activité 10](#)

Authentification (API REST sécurisée par JWT)

- Inscription : `POST /api/auth/register` → renvoie un JWT.
- Connexion : `POST /api/auth/login` → renvoie un JWT.
- Identité : `GET /api/auth/me` (JWT requis).
- Les routes de jeu exigent l'en-tête `Authorization: Bearer <token>` (401 sinon).

- Mots de passe **hachés bcrypt**, jetons **signés HS256**, rôles **player / admin**.
- Créer un admin : `python -m backend.create_admin <user> <mot_de_passe>`.

Dossier RNCP / soutenance

Un dossier de preuves a été ajouté pour relier le projet à la grille **Expert en développement logiciel RNCP 39583** :

- [Dossier final Bloc 2 — rendu 24/07/2026](#)
- [Annexes Bloc 2 — preuves hors limite de pages](#)
- [Cartographie avec la grille](#)
- [Bloc 1 — cadrage projet](#)
- [Bloc 2 — conception et développement](#)
- [Bloc 3 — pilotage projet](#)
- [Bloc 4 — maintenance](#)
- [Plan de recette](#)
- [Support de soutenance](#)
- [Checklist jury](#)
- [Architecture BDD et multi-utilisateur](#)
- [Déploiement VPS Docker](#)
- [Kanban projet](#)
- [Stratégie Git, branches, tags et pipeline](#)

État des attendus complémentaires

Attendu	Statut	Preuve
BDD	En place	SQLite via backend/database.py
Authentification JWT	En place	backend/auth.py + écran de connexion frontend
Multi-utilisateur	En place	Identité issue du JWT, sauvegardes isolées par compte
Pipeline CI/CD	En place	CI GitHub Actions , déploiement VPS + GitLab CI
Tests unitaires frontend	En place	Vitest + React Testing Library dans frontend/src/components/tests

Attendu	Statut	Preuve
Tests end-to-end	En place	Playwright dans frontend/e2e/game.spec.js
Déploiement VPS	En place	Docker Compose + guide docs/deploiement_vps.md
Kanban	En place	docs/gestion_projet/kanban.md
Git tag / multibranche	Prévu dans Git	docs/gestion_projet/strategie_git.md

Plan d'implementation

1. **Prompt & lore** a part : `prompt_survivant.md` fixe la voix du MJ, les jauges et les consignes grimdark.
2. **Donnees joueur** : `character_sheet.yaml` contient attributs, ressources, stress.
3. **Scripts** : `run_session.py` charge prompt + fiche, initialise la conversation et gere les commandes (lancer de des, affichage d'etat, sauvegardes, sortie).
4. **Lib Python** : utilitaires dans `src/` (gestion d'etat, formatage du prompt, tirages 2d6).
5. **Tests** : `pytest` valide chargement de fiche et coherence des jets.
6. **Securite** : la clef OpenAI reste dans l'env (`OPENAI_API_KEY`). Exemple : `.env.example`.

Prerequis

- Python 3.11+
- Compte OpenAI avec acces aux modeles GPT compatibles chat
- Clef API stockee dans une variable d'environnement `OPENAI_API_KEY`

Installation

```
cd "c:\Users\ps3ka\OneDrive\Bureau\rpg 40k"
python -m venv .env
.\.env\Scripts\Activate.ps1
pip install -r requirements.txt
```

Copiez `.env.example` en `.env` puis remplissez votre clef.

Déploiement VPS

Le projet peut être déployé sur un VPS Linux avec Docker Compose :

```
cd /opt/rpg-40k
docker compose up -d --build
```

Le guide complet est disponible ici : [docs/deploiement_vps.md](https://docs.rpg-40k.com/deploiement_vps.md).

Lancer une session

```
.\.venv\Scripts\Activate.ps1
python run_session.py --model gpt-4.1-mini
```

Commandes disponibles pendant la session :

- `!roll` pour lancer 2d6 localement
- `!status` pour afficher la fiche actuelle
- `!save` pour forcer une sauvegarde meme si l'auto-save est inactive
- `!help` pour recapitulatif
- `!quit` pour terminer

Sauvegardes par scene

Le script cree automatiquement un snapshot apres chaque scene

(`saves/scene_###.yaml` par default). Options utiles :

```
python run_session.py --save-dir parties\aelia --save-format json --no-auto-save
```

- `--save-dir` personnalise le dossier
- `--save-format` accepte `yaml` (default) ou `json`

- `--no-auto-save` desactive les snapshots automatiques; utilisez `!save` pour capturer une scene manuelle

Tests unitaires

```
.\.venv\Scripts\Activate.ps1  
pytest
```

Les tests couvrent la creation de fiches et la distribution des jets 2d6.

Tests unitaires frontend

```
cd frontend  
npm test
```

Les tests unitaires frontend utilisent Vitest, jsdom et React Testing Library.

Tests end-to-end

Les tests navigateur utilisent Playwright.

```
cd frontend  
npm run e2e:install  
npm run e2e
```

Ils necessitent que le backend FastAPI soit lance sur `127.0.0.1:8000` et le frontend Vite sur `127.0.0.1:5173`.

Structure

```
README.md  
prompt_survivant.md  
character_sheet.yaml
```

```
run_session.py
src/
  __init__.py
  dice.py
  state.py
  prompt_builder.py
requirements.txt
.env.example
tests/
  test_state.py
  test_dice.py
```

Mentions importantes

- Le projet fournit un cadre narratif grimdark mais n'utilise aucun contenu officiel protégé.
- Ne partagez jamais votre clé API publiquement; utilisez `.env` ou le gestionnaire de secrets de votre OS.
- Adaptez librement les attributs et ressources pour varier vos parties.